Anika Bhattacharya
Travis Gillespie
Marty Yossifov
William Zou

DSC232R

Professor Edwin Solares
TA Nishanth Ramesha

1) c) https://huggingface.co/datasets/mercari-us/merrec
2) SDSC Expanse Environment Setup
   a) We use the SDSC Expanse portal to launch a jupyter environment for running Spark workloads. The benefit is the HPC resources' processing power over that of one's local machine. We choose the following environment settings:
      Partition: shared
      Time limit: 180 minutes
      Number of cores: 64
      Memory required per node: 128 GB
      Working Directory: home
      Environment Modules to be loaded: singularitypro
      Type: JupyterLab
      Singularity Image File Location:
      ~/esolares/singularity_images/spark_py_latest_jupyter_dsc232r.sif

   b) spark = SparkSession.builder \
      .config("spark.driver.memory", "2g") \
      .config("spark.executor.memory", "2g") \
      .config("spark.executor.instances", 63) \
      .getOrCreate()
      Reasoning: The data set being used is 166gb so to speed up processing we use a relatively large number of cores for increased parallelism. As long as we avoid using memory intensive operations such as .collect() 128 gb of memory is enough to handle operations on a file this large without crashing because not all of it is on the RAM at any given point.
   c) Using `Executor instances = Total Cores - 1` and `Executor memory = (Total Memory - Driver Memory) / Executor Instances` we get 63 executor instances = 64-1 and executor memory = (128-2)/63 = ~2 GB.

[13]:
```python
import requests
import pandas as pd

# Get the active Spark Context and URL
sc = spark.sparkContext
url = f"{sc.uiWebUrl}/api/v1/applications/{sc.applicationId}/executors"

# Fetch the executor data from the API
response = requests.get(url)
executors = response.json()

# Format into a readable DataFrame
df = pd.DataFrame(executors)[['id', 'totalCores', 'maxMemory', 'activeTasks', 'isActive']]
df['maxMemory_GB'] = (df['maxMemory'] / (1024**3)).round(2)
df
```

[13]:

	id	totalCores	maxMemory	activeTasks	isActive	maxMemory_GB
0	driver	64	1099746508	0	True	1.02

d)

3.) Data Exploration using Spark

a.)

There are 1,268,092,738 observations in the training set.

b.)

User_id (categorical): Integer that identifies a user, unique to every user.
Stime (continuous): Timestamp(utc) indicating when an event occurred.
Session_id (categorical): 64 character string that identifies a user's session, groups of events from a user in a short time period are grouped into a single session.
Sequence_id (categorical): String that represents an event's date and which datashard it belongs to.
Sequence_length (categorical): Integer that denotes the number of events that happened within a session.
Event_id (categorical): String that indicates the type of event that occurred. Uses 6 possible string values (item_view, item_like, item_comment, item_add_to_cart, item_purchase, item_share) to represent what was the user's event. Highly imbalanced towards item_viewed and item_liked. The target column of this dataset.
Item_id (categorical): Integer that represents an item ID unique to a single listing post.
Product_id (categorical): String that is the concatenation of a product's brand ID and c2 ID.
Name (text) : The product's listing title, slightly right-skewed distribution.
Price (continuous): Float that represents the price of the item listed is US Dollars, highly right-skewed distribution.
C0_name (low cardinality categorical): String that represents what broad category a listing's product belongs to with 17 classes.
C0_id (low cardinality categorical): Integer representation of C0_name classes.

C1_name (high cardinality categorical): String that narrows the listing's description from the broad C0 classes.
C1_id (high cardinality categorical): Integer representation of C1_name classes.
C2_name (high cardinality categorical): String that further narrows the product's description from the C1 classes.
C2_id (high cardinality categorical): Integer representation of C2_name classes.
Brand_name (categorical): String that is the text label for a listed item's brand.
Brand_id (categorical): Integer representation of brand_name that uniquely identifies it.
Item_condition_name (categorical): String describing the item's condition using one of five different categories (Good, New, Like new, Fair, Poor).
Item_condition_id (categorical): Integer representation of Item_condition_name classes.
Size_name (categorical): String representing the size of the listed item. Imbalanced towards null values due to most items not having size information.
Size_id (categorical): Integer representation of size_name. Imbalanced towards null values due to most items not having size information.
Color (categorical): String describing the color of the list item. Imbalanced towards null values due to most items not having color information.
Shipper_name (categorical): String that has two classes (buyer, seller) that denotes which of the two pays for shipment.
Shipper_id (categorical): Integer representation of shipper_name.
c.) Yes, there are missing values in several of the columns/categories. The columns with missing data include: **brand_name, size_name, color, c1_name, c2_name, item_condition_name.** These missing entries likely happen because product listings are usually not fully populated by users, which causes missing fields.

user_id	stime	session_id	sequence_id	sequence_length	event_id	item_id	product_id	name	price	c0_name	c0_id	c1_name	c1_id	c2_name	c2_id	brand_name	brand_id	item_condition_id	item_condition_name	size_name	size_id	color	shipper_id	shipper_name
0	0	0	0	0	0	0	0	0	0	0	0	239418	239418	79007951	79007951	234303407	0	0	500309	792001318	792001318	12140645598	0	0

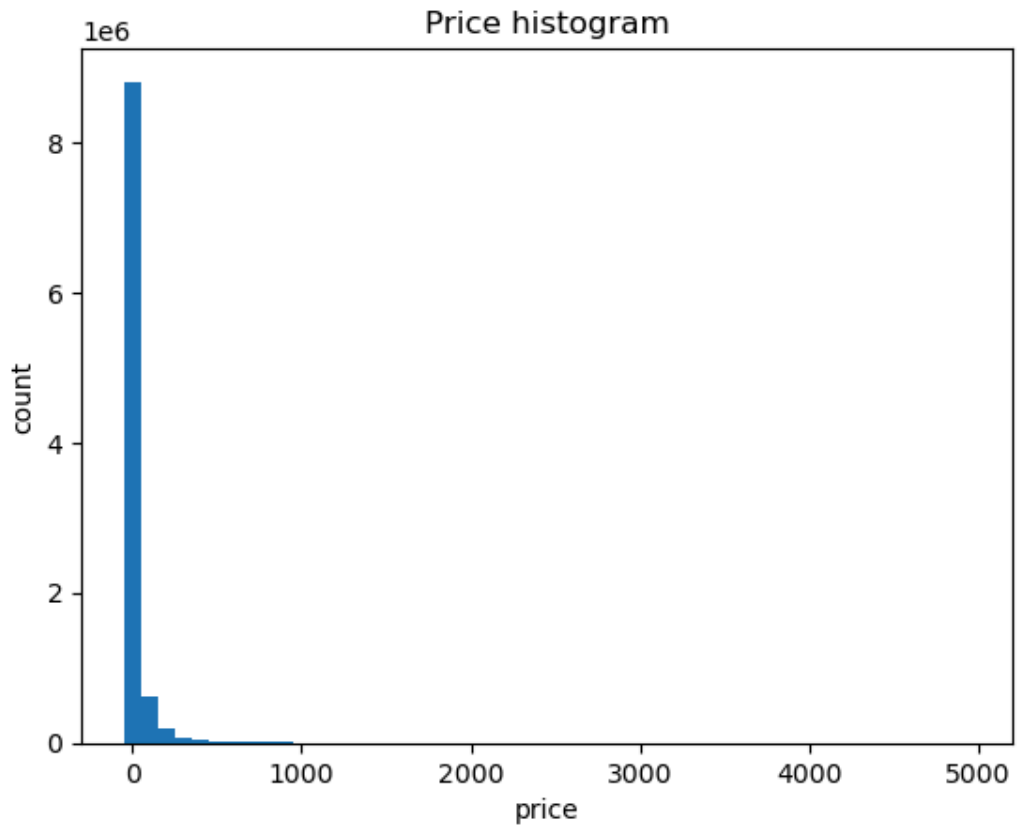
c1_name & c1_id have 239418 missing values
c2_name & c2_name have 79007951 missing values
brand_name has 234303407 missing values
size_name & size_id have 792001318 missing values
color has 12140645598 missing values
item_condition_name has 500309 missing values

There are no duplicates in the data; this confirmed by the research paper associated with the dataset; "the dataset, in addition to standard attributes like user_id, item_id, and session_id, also incorporates unique features such as timestamped action types, detailed product taxonomy, and textual product attributes". Because of the unique nature of the timestamp in stime, there is no possibility for a duplicate entry. Additionally, the paper also reviews the data cleaning and processing steps, explicitly removing duplicates: "Redundancy Reduction: To address the issue of repetitive actions within sequences, such as consecutive clicks on the same item, a

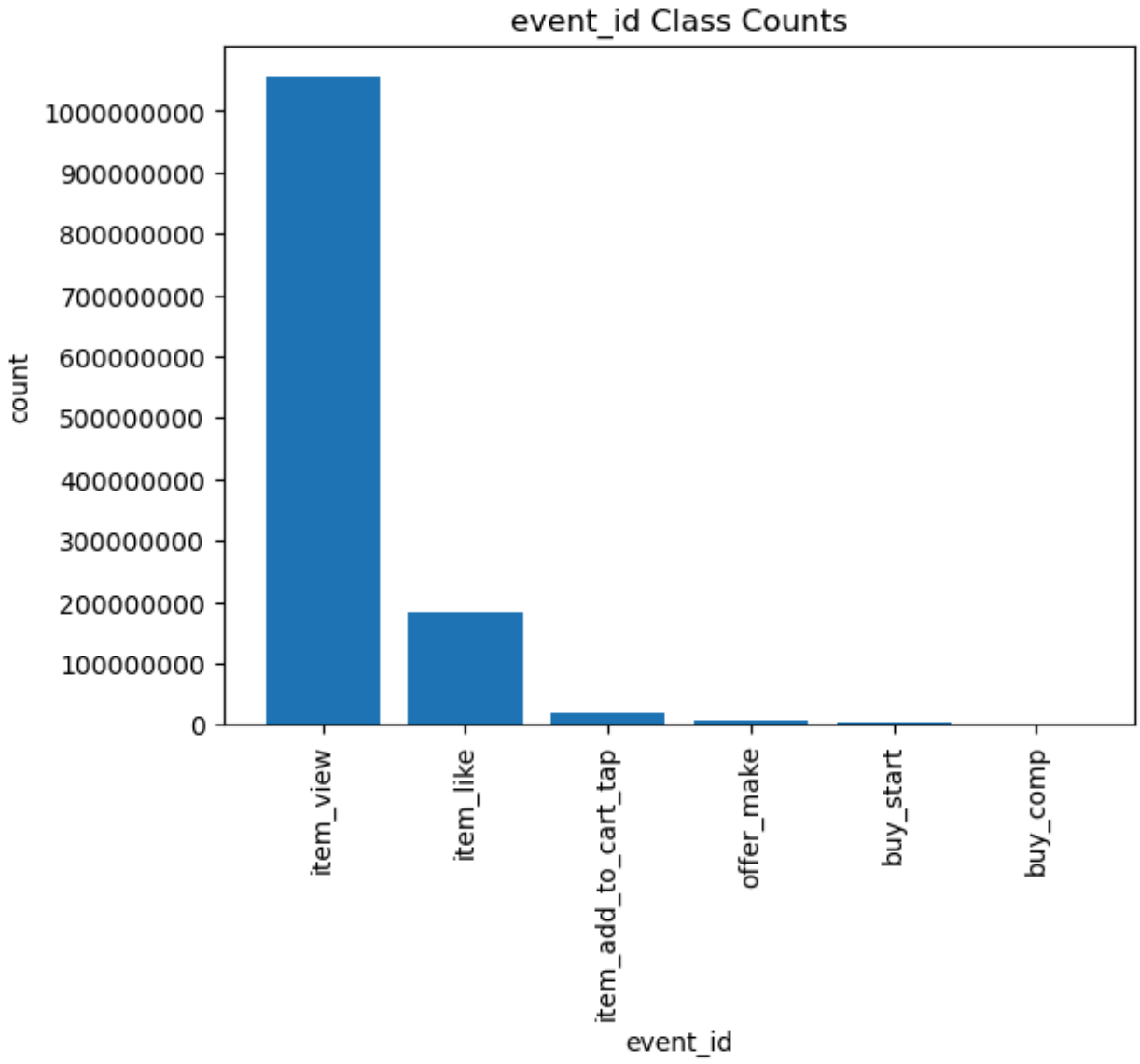
deduplication process was applied. This step ensured that only unique consecutive interactions were retained, reducing redundancy and enhancing data conciseness.”

4) Data Plots

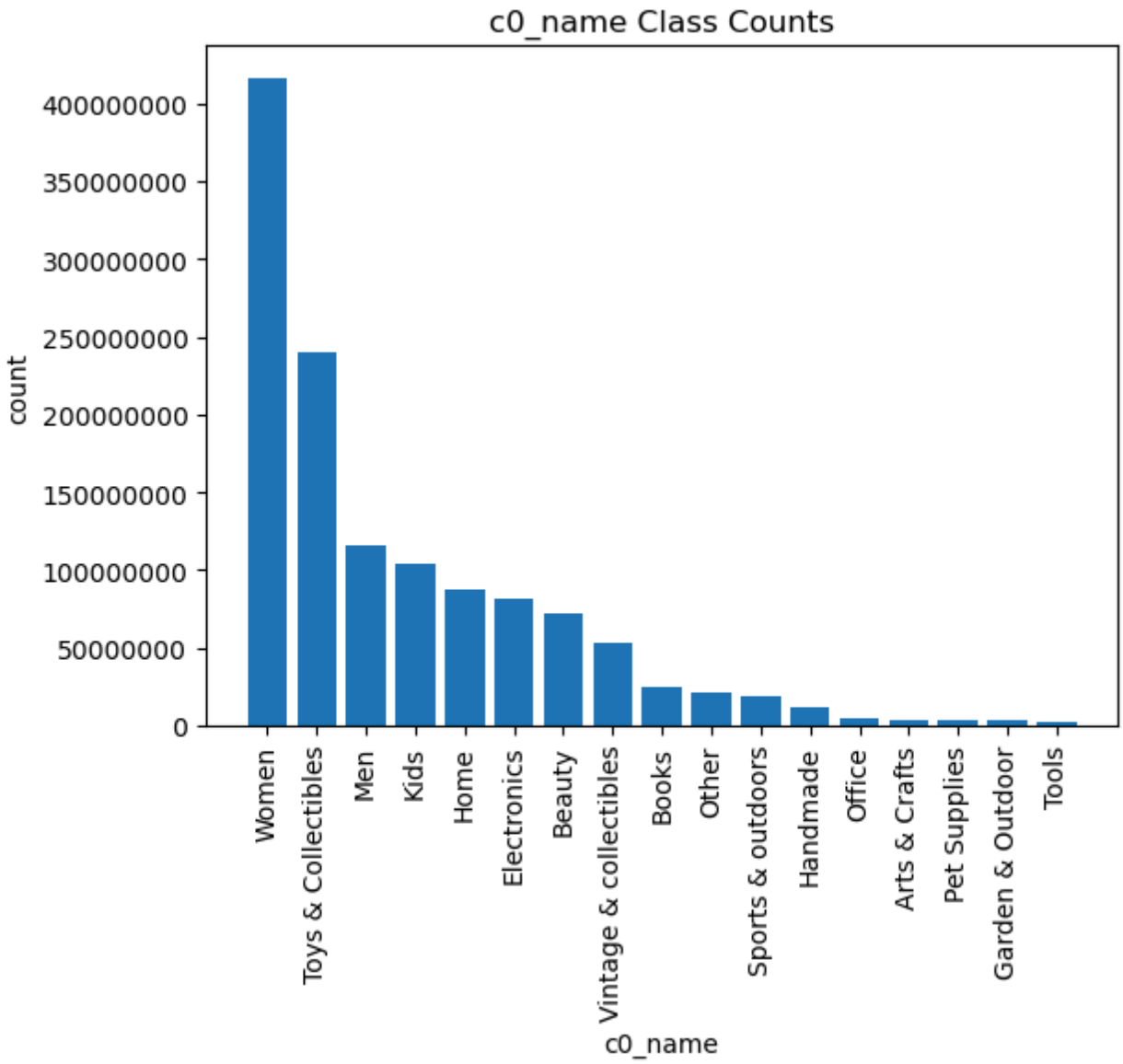
Plot 1



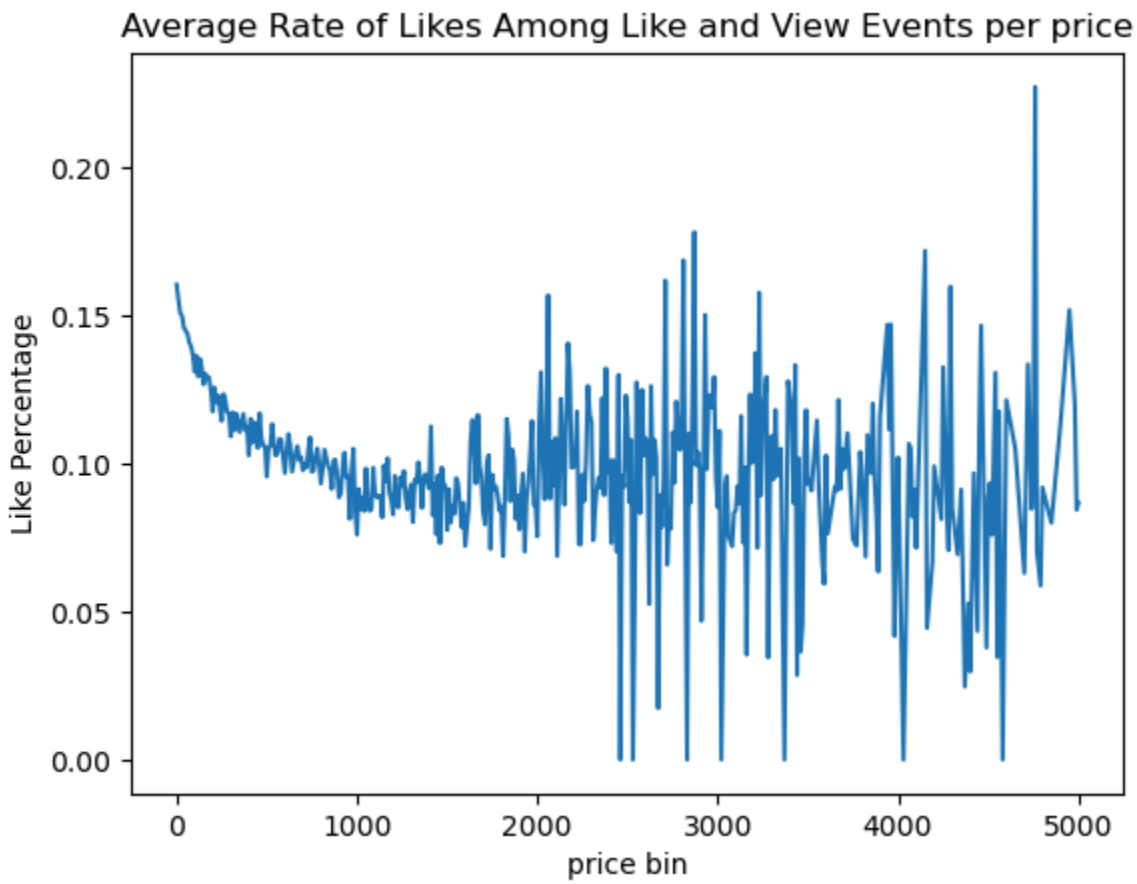
Plot 2


event_id Class Counts
count
1000000000
900000000
800000000
700000000
600000000
500000000
400000000
300000000
200000000
100000000
0
item_view
item_like
item_add_to_cart_tap
offer_make
buy_start
buy_comp
event_id

Plot 3


c0_name Class Counts
count
400000000
350000000
300000000
250000000
200000000
150000000
100000000
50000000
0
Women
Toys & Collectibles
Men
Kids
Home
Electronics
Beauty
Vintage & collectibles
Books
Other
Sports & outdoors
Handmade
Office
Arts & Crafts
Pet Supplies
Garden & Outdoor
Tools
c0_name

Plot 4



c) **Plot 1** is a histogram that shows the distribution of item prices across the dataset. The distribution is heavily skewed to the right, as most items are priced near $0 - $50, with the most common price range being the lowest. The prices for items extend all the way up to $5000, showing the small amount of high-value items.

Plot 2 is a bar chart that shows the count of each user interaction event type (**event_id**) across the dataset. **item_view** is by far the most common event, with over 1 billion occurrences, indicating that the majority of user interactions are just passively browsing, where users look/click on items, rather than acting on them. Item_like is the second most frequent event, at around 180 million occurrences, showing that saving or favoriting an item is the next most common behavior. The following events, such as item_add_to_cart_tap (25 million), offer_make (8 million), buy_start (3 million), and buy_comp (1 million), show a steep drop-off, which show that the vast majority of users browse rather than purchase or act on items.

Plot 3 is a bar chart that shows the events broken down by the product category, stored in **c0_name.** Women's fashion is the highest populated category with over ~400 million events, nearly double the next category after, which is Toys & Collectibles at ~240 million. Men, Kids, Home, Electronics, and Beauty form a mid-tier cluster at about 80-120 million events each. Finally, categories like Handmade, Office, Arts & Crafts, Pet Supplies, Garden & Outdoor, Tools, all have low engagement, suggesting a lower amount of items are listed under these categories, as well as less engagement from the general population, suggesting niche audiences. The category imbalance means that any category-aware model will need to handle the underrepresented classes by possibly implementing stratification.

Plot 4 is a line graph that plots the average rate of like events among like and view events relative to prices for the dataset's listings that have been grouped into 10 dollar bins. This graph shows that there is an approximately exponential decay in like events relative to like and view events until about the 2000 dollar bin. After the 2000 dollar buck the graph has much more variance and doesn't show much change in the like average as price increases. This is likely due to high price items being much less common than low price items making price buckets past 2000 dollars more vulnerable to randomness and noise.

5) Preprocessing plan

Handling missing values: we will assess missing values across all the columns in the MerRec dataset using Spark operations like .na.drop() and .describe(). For numeric fields, we intend to use median imputation to reduce the impact of outliers and for categorical fields, we plan to use a placeholder like "Unknown".

Handling data imbalance: We will likely have to undersample the common classes such as item_viewed and item_liked while oversampling the item_purchase class to prevent models from ignoring the rarer classes and still getting high accuracy.

Transformations: We will use feature engineering to create a day of the week variable from the stime variable to see if it is more predictive than just the date. The price variable will be scaled from 0 to 1 based on the minimum and maximum values of price. One hot encoding will be used to turn categorical variables with classes such as our target variable event into binary variables representing each class to make it readable and usable for machine learning.

Spark Operations: We will load the data using spark.read.parquet() (or equivalent) with a predefined schema, then filter relevant interaction types using filter() and clean the data with dropna(), fillna(), and dropDuplicates(). To capture temporal user behavior, we will construct sequence-based features using window operations (Window.partitionBy().orderBy()), along with functions such as lag(), lead(), and withColumn(), and compute aggregates using groupBy().agg(). We will transform data using select(), cast(), and text processing (e.g., tokenization and TF-IDF) will be applied. Finally, we will optimize performance using repartition(), cache(), and broadcast() joins, and write the processed dataset using write.parquet() for efficient downstream modeling.